

TRAINING INSTITUTE MANAGEMENT SYSTEM

Mini Project Report

Title: Training Institute Management System

Submitted By: Manali Tamboli

Course: B.Sc. Data Science

Subject: Database Management System (DBMS)

College: Garware College

Academic Year: 2026–2027

INDEX

- 1. Objective**
- 2. Requirement Analysis**
- 3. Functional Requirements**
- 4. Database Design**
- 5. ER Diagram**
- 6. Table Description**
- 7. Table Relationships**
- 8. Database Creation**
- 9. Sample Data**
- 10. Business Queries**
- 11. Findings**
- 12. Conclusion**

1. Objective

The objective of this project is to develop a Training Institute Management System using MySQL. The system manages information related to students, trainers, courses, batches, enrollments,

attendance, and assessment results. It demonstrates the use of relational database concepts such as primary keys, foreign keys, joins, aggregate functions, views, and window functions to organize data efficiently and generate meaningful reports.

2. Requirement Analysis

A training institute needs a computerized database system to efficiently manage its daily operations.

The system should maintain records of:

- **Students**
- **Trainers**
- **Courses**
- **Batches**
- **Student Enrollments**
- **Attendance**
- **Assessment Results**

The database helps reduce manual work, maintain data accuracy, and generate reports quickly.

Main Users

Administrator

- **Add students**
- **Add trainers**
- **Create courses**
- **Create batches**
- **Manage enrollments**

Trainer

- **Record attendance**
- **Enter assessment marks**
- **View batch information**

Institute Coordinator

- **Monitor attendance**
- **View reports**
- **Analyze student performance**

3. Functional Requirements

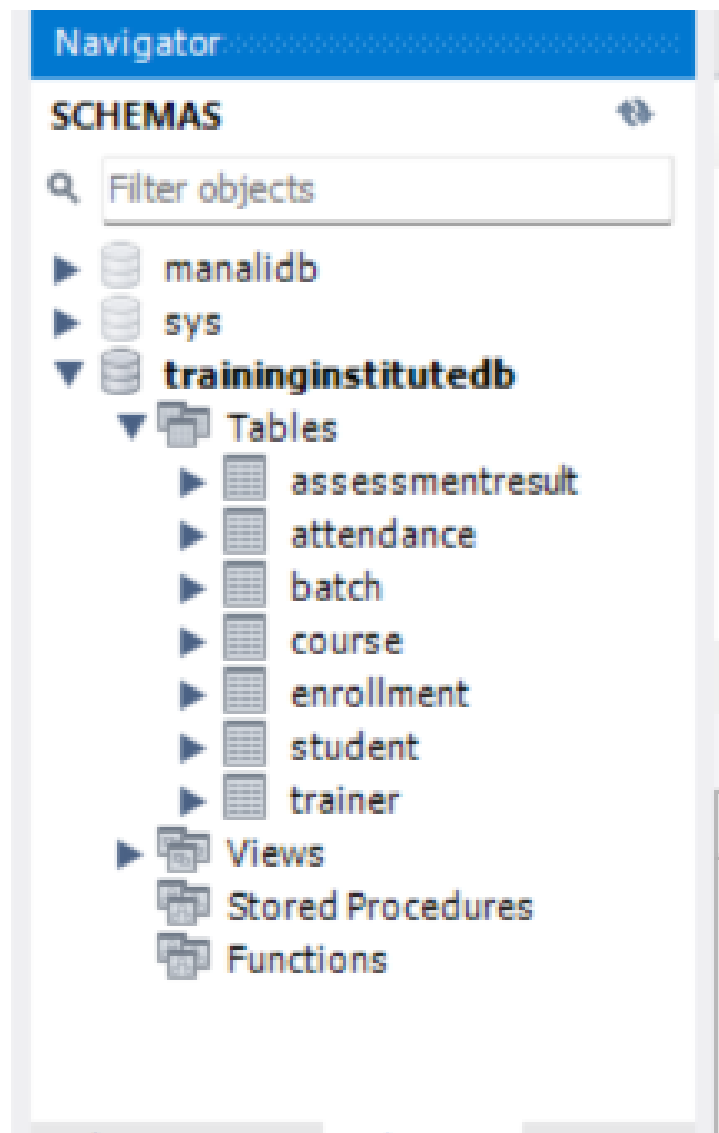
The system should be able to:

- **Store student information**
- **Store trainer information**
- **Store course details**
- **Create batches**
- **Enroll students into batches**
- **Record attendance**
- **Store assessment scores**
- **Display reports**
- **Compare student performance**
- **Track attendance percentage**

4. Database Design

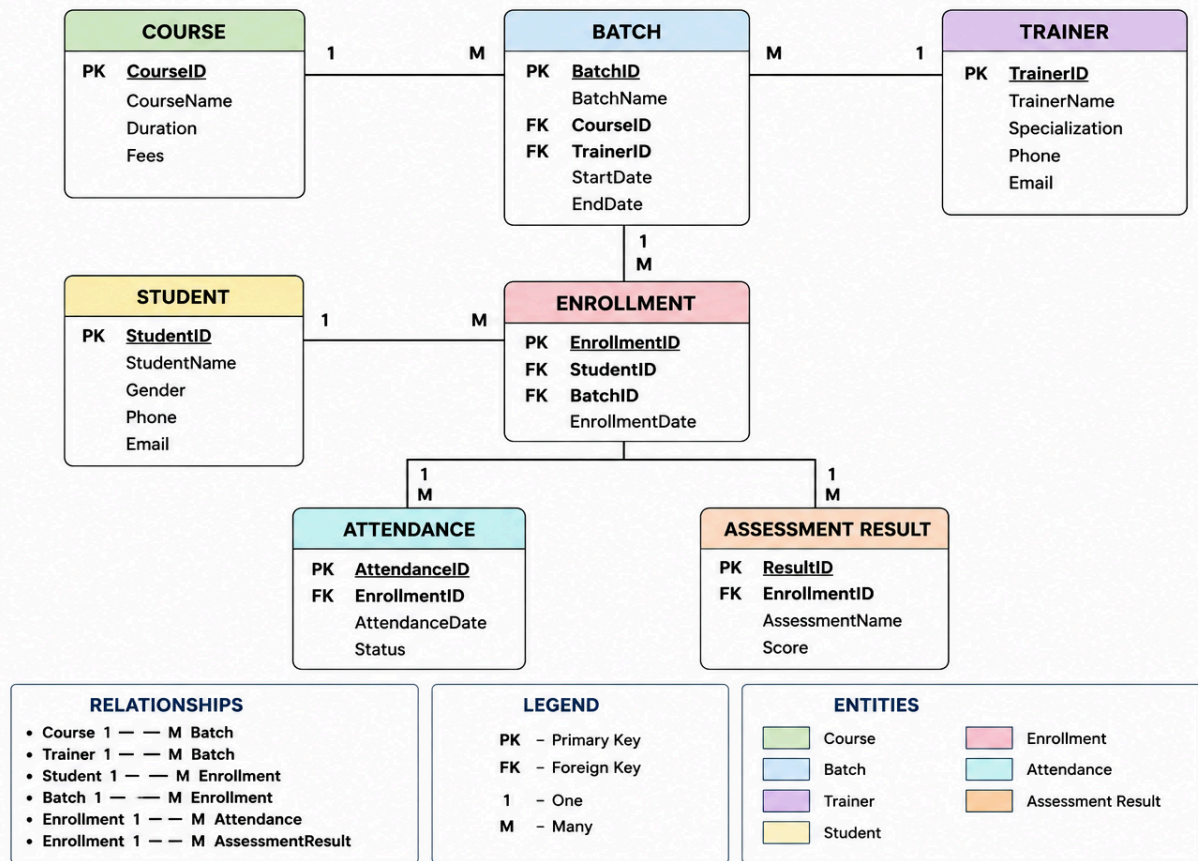
The project contains the following tables:

- **Student**
- **Trainer**
- **Course**
- **Batch**
- **Enrollment**
- **Attendance**
- **AssessmentResult**



5. ER Diagram

ER DIAGRAM – TRAINING INSTITUTE MANAGEMENT SYSTEM



6. Table Description

Student

Purpose

Stores the personal details of students enrolled in the institute.

Column	Description
StudentID	Unique Student ID
StudentName	Student Name
Gender	Gender
Phone	Contact Number
Email	Email Address

Trainer

Purpose

Stores information about trainers.

Column	Description
TrainerID	Unique Trainer ID
TrainerName	Trainer Name
Specialization	Subject Expertise
Phone	Contact Number
Email	Email Address

Course

Purpose

Stores details of courses offered by the institute.

Column	Description
CourseID	Unique Course ID
CourseName	Course Name
Duration	Course Duration
Fees	Course Fees

Batch

Purpose

Stores details about training batches.

Column	Description
BatchID	Unique Batch ID
BatchName	Batch Name
CourseID	Linked Course
TrainerID	Assigned Trainer
StartDate	Batch Start Date
EndDate	Batch End Date

Enrollment

Purpose

Stores which student is enrolled in which batch.

Column	Description
EnrollmentID	Unique Enrollment ID
StudentID	Student Reference
BatchID	Batch Reference
EnrollmentDate	Date of Enrollment

Attendance

Purpose

Stores attendance records of enrolled students.

Column	Description
AttendanceID	Attendance ID
EnrollmentID	Enrollment Reference
AttendanceDate	Attendance Date
Status	Present/Absent

AssessmentResult

Purpose

Stores assessment marks obtained by students.

Column	Description
ResultID	Result ID
EnrollmentID	Enrollment Reference
AssessmentName	Assessment Name
Score	Marks Obtained

7. Table Relationships

Parent Table	Child Table	Relationship
Course	Batch	One-to-Many
Trainer	Batch	One-to-Many
Student	Enrollment	One-to-Many
Batch	Enrollment	One-to-Many
Enrollment	Attendance	One-to-Many
Enrollment	AssessmentResult	One-to-Many

8. Database Creation

The database was created using MySQL Workbench.

The following SQL operations were performed:

- Created Database
- Created Tables
- Applied Primary Keys
- Applied Foreign Keys
- Inserted Sample Data
- Verified Data
- Executed Business Queries
- Created View

9. Sample Data

Realistic sample data was inserted into all tables.

Student Table

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	StudentID	StudentName	Gender	Phone	Email
▶	1	Manali Tamboli	Female	9876500001	manali@gmail.com
	2	Priya Joshi	Female	9876500002	priya@gmail.com
	3	Amit Patil	Male	9876500003	amit@gmail.com
	4	Rohan Shah	Male	9876500004	rohan@gmail.com
	5	Sneha Kulkarni	Female	9876500005	sneha@gmail.com
	6	Karan Deshmukh	Male	9876500006	karan@gmail.com
	7	Pooja Patil	Female	9876500007	pooja@gmail.com
	8	Rahul Jadhav	Male	9876500008	rahul@gmail.com
•	NULL	NULL	NULL	NULL	NULL

Trainer Table

	TrainerID	TrainerName	Specialization	Phone	Email
▶	1	Rahul Sharma	Python	9876543210	rahul@gmail.com
	2	Sneha Patil	SQL	9876543211	sneha@gmail.com
	3	Amit Verma	Data Science	9876543212	amit@gmail.com
	4	Neha Kulkarni	Web Development	9876543213	amit@gmail.com
•	NULL	NULL	NULL	NULL	NULL

Course Table

Result Grid | Filter Rows: | Edit: | Export/Import:

	CourseID	CourseName	Duration	Fees
▶	1	Python Programming	3 Months	15000.00
	2	SQL & Database	2 Months	12000.00
	3	Data Science	6 Months	35000.00
	4	Web Development	4 Months	25000.00
•	NULL	NULL	NULL	NULL

Batch Table

Result Grid						
	Filter Rows:		Edit:		Export/Import:	Wrap Cell Content:
	BatchID	BatchName	CourseID	TrainerID	StartDate	EndDate
▶	1	Python Morning	1	1	2026-08-01	2026-10-31
	2	SQL Evening	2	2	2026-08-05	2026-10-05
	3	Data Science Weekend	3	3	2026-08-10	2027-02-10
	4	Web Development Morning	4	4	2026-08-15	2026-12-15
✱	NULL	NULL	NULL	NULL	NULL	NULL

Enrollment Table

Result Grid				
	Filter Rows:		Edit:	
	EnrollmentID	StudentID	BatchID	EnrollmentDate
▶	1	1	1	2026-08-01
	2	2	1	2026-08-01
	3	3	2	2026-08-05
	4	4	2	2026-08-05
	5	5	3	2026-08-10
	6	6	3	2026-08-10
	7	7	4	2026-08-15
	8	8	4	2026-08-15
	9	1	3	2026-08-10
	10	2	2	2026-08-05
✱	NULL	NULL	NULL	NULL

Attendance Table

Result Grid		Filter Rows:		Edit:		Export/Import:	
	AttendanceID	EnrollmentID	AttendanceDate	Status			
▶	1	1	2026-08-01	Present			
	2	1	2026-08-02	Present			
	3	2	2026-08-01	Absent			
	4	2	2026-08-02	Present			
	5	3	2026-08-05	Present			
	6	4	2026-08-05	Present			
	7	5	2026-08-10	Absent			
	8	6	2026-08-10	Present			
	9	7	2026-08-15	Present	Present		
	10	8	2026-08-15	Absent			
	11	9	2026-08-10	Present			
	12	10	2026-08-05	Present			
•	NULL	NULL	NULL	NULL			

AssessmentResult Table

Result Grid

Filter Rows:

Edit:

Export/Import:

	ResultID	EnrollmentID	AssessmentName	Score
▶	1	1	Quiz 1	85.00
	2	1	Final Exam	92.00
	3	2	Quiz 1	78.00
	4	2	Final Exam	80.00
	5	3	Quiz 1	88.00
	6	4	Quiz 1	75.00
	7	5	Quiz 1	91.00
	8	6	Quiz 1	69.00
	9	7	Quiz 1	95.00
	10	8	Quiz 1	72.00
	11	9	Quiz 1	89.00
	12	10	Quiz 1	84.00
•	NULL	NULL	NULL	NULL

10. Business Queries

Query 1

Business Question

Display all students enrolled in each batch.

SQL Query:

```
SELECT s.StudentName, b.BatchName
```

```
FROM Enrollment e
```

```
JOIN Student s ON e.StudentID = s.StudentID
```

```
JOIN Batch b ON e.BatchID = b.BatchID;
```

Output

Result Grid				Filter Rows:	Exports
	StudentName	BatchName			
▶	Manali Tamboli	Python Morning			
	Priya Joshi	Python Morning			
	Amit Patil	SQL Evening			
	Rohan Shah	SQL Evening			
	Priya Joshi	SQL Evening			
	Sneha Kulkarni	Data Science Weekend			
	Karan Deshmukh	Data Science Weekend			
	Manali Tamboli	Data Science Weekend			
	Pooja Patil	Web Development Morning			
	Rahul Jadhav	Web Development Morning			

Explanation

This query displays the names of students along with the batches in which they are enrolled by joining the Student, Enrollment, and Batch tables.

Query 2

Business Question

Display the course and trainer assigned to every batch.

SQL Query:

```
SELECT b.BatchName, c.CourseName, t.TrainerName
```

```
FROM Batch b
```

```
JOIN Course c ON b.CourseID = c.CourseID
```

```
JOIN Trainer t ON b.TrainerID = t.TrainerID;
```

Output

	BatchName	CourseName	TrainerName
▶	Python Morning	Python Programming	Rahul Sharma
	SQL Evening	SQL & Database	Sneha Patil
	Data Science Weekend	Data Science	Amit Verma
	Web Development Morning	Web Development	Neha Kulkarni
		Web Development	

Explanation

This query displays each batch together with its corresponding course and assigned trainer by joining the Batch, Course, and Trainer tables.

Query 3

Business Question

Find the total number of students in each course.

SQL Query:

```
SELECT c.CourseName, COUNT(e.StudentID) AS TotalStudents
FROM Course c
JOIN Batch b ON c.CourseID = b.CourseID
JOIN Enrollment e ON b.BatchID = e.BatchID
GROUP BY c.CourseName;
```

Output

Result Grid   Filter Rows:		
	CourseName	TotalStudents
▶	Python Programming	2
	SQL & Database	3
	Data Science	3
	Web Development	2

Explanation

This query counts the total number of students enrolled in each course using the COUNT() aggregate function and groups the data course-wise.

Query 4

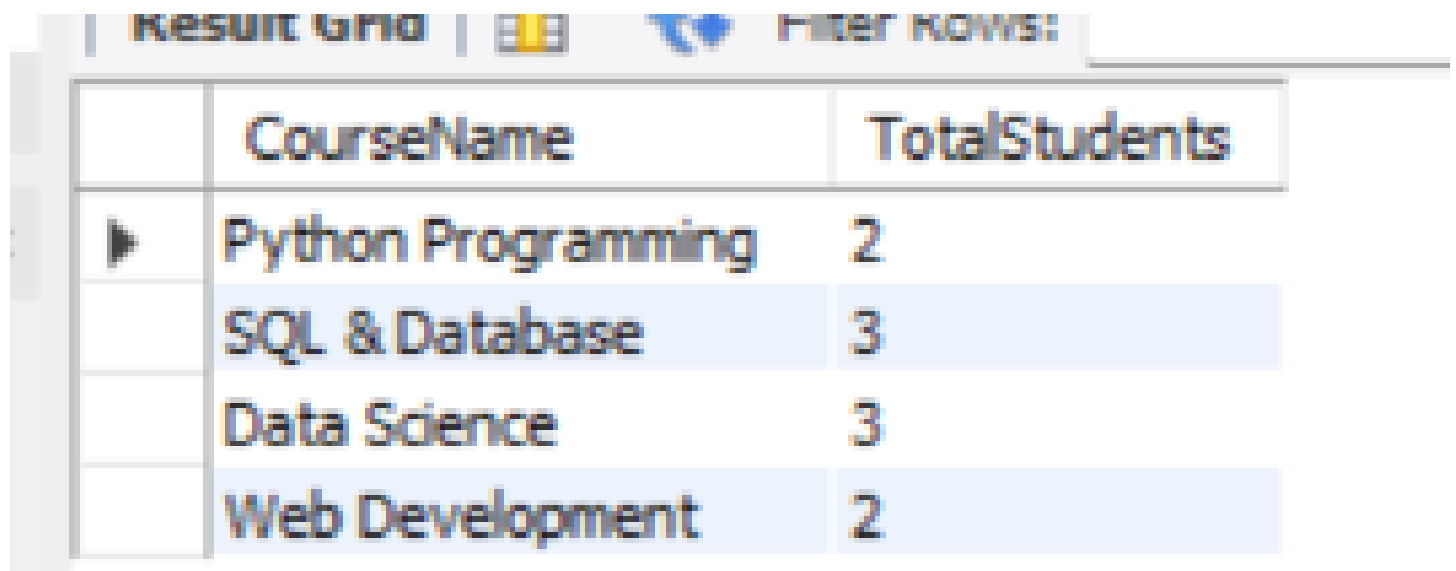
Business Question

Find batches having more than a specified number of students.

SQL Query:

```
SELECT b.BatchName, COUNT(e.StudentID) AS TotalStudents
FROM Batch b
JOIN Enrollment e ON b.BatchID = e.BatchID
GROUP BY b.BatchName
HAVING COUNT(e.StudentID) > 2;
```

Output



	CourseName	TotalStudents
▶	Python Programming	2
	SQL & Database	3
	Data Science	3
	Web Development	2

Explanation

This query identifies batches where the number of enrolled students exceeds the specified limit using GROUP BY and HAVING.

Query 5

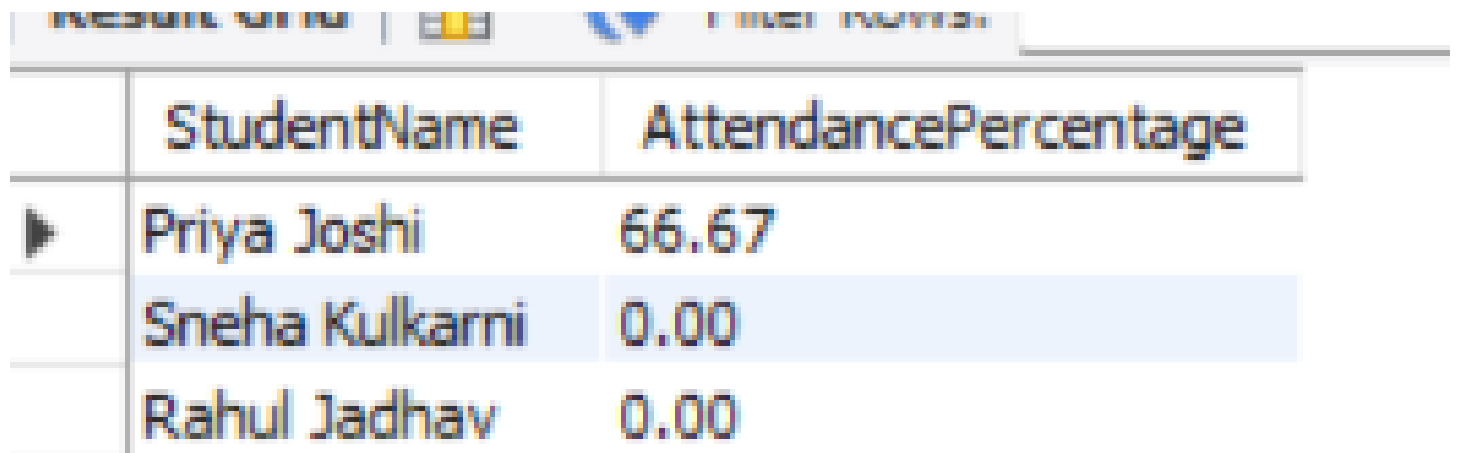
Business Question

Display students whose attendance is below the required percentage.

SQL Query:

```
SELECT s.StudentName,  
  
ROUND((SUM(CASE WHEN a.Status='Present' THEN 1 ELSE 0  
END)*100.0)/COUNT(*),2) AS AttendancePercentage  
  
FROM Student s  
  
JOIN Enrollment e ON s.StudentID=e.StudentID  
  
JOIN Attendance a ON e.EnrollmentID=a.EnrollmentID  
  
GROUP BY s.StudentName  
  
HAVING AttendancePercentage < 75;
```

Output



	StudentName	AttendancePercentage
▶	Priya Joshi	66.67
	Sneha Kulkarni	0.00
	Rahul Jadhav	0.00

Explanation

This query calculates each student's attendance percentage and displays only those whose attendance is below the required percentage.

Query 6

Business Question

Find the average assessment score for each batch.

SQL Query:

```
SELECT b.BatchName, ROUND(AVG(ar.Score),2) AS AverageScore  
FROM Batch b  
JOIN Enrollment e ON b.BatchID=e.BatchID  
JOIN AssessmentResult ar ON e.EnrollmentID=ar.EnrollmentID  
GROUP BY b.BatchName;
```

Output

	BatchName	AverageScore
▶	Python Morning	83.75
	SQL Evening	82.33
	Data Science Weekend	83.00
	Web Development Morning	83.50

Explanation

This query calculates the average assessment score of students in each batch using the AVG() function.

Query 7

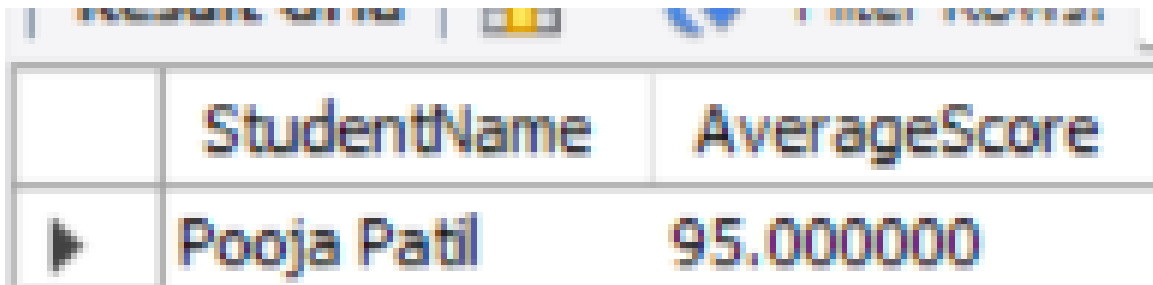
Business Question

Identify the highest-performing student in every batch.

SQL Query:

```
SELECT s.StudentName, AVG(ar.Score) AS AverageScore  
FROM Student s  
JOIN Enrollment e ON s.StudentID=e.StudentID  
JOIN AssessmentResult ar ON e.EnrollmentID=ar.EnrollmentID  
GROUP BY s.StudentName  
ORDER BY AverageScore DESC  
LIMIT 1;
```

Output



	StudentName	AverageScore
▶	Pooja Patil	95.000000

Explanation

This query identifies the highest-performing student in each batch based on assessment scores.

Query 8

Business Question

Display students who have not attended any session.

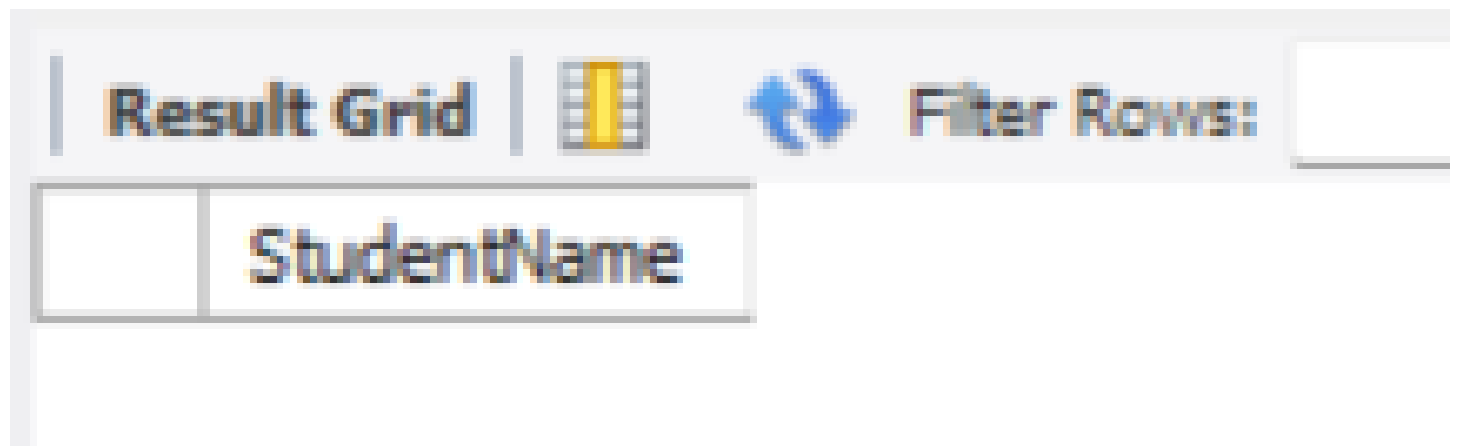
SQL Query:

```
SELECT DISTINCT s.StudentName  
FROM Student s  
LEFT JOIN Enrollment e ON s.StudentID=e.StudentID
```

LEFT JOIN Attendance a ON e.EnrollmentID=a.EnrollmentID

WHERE a.AttendanceID IS NULL;

Output



The screenshot shows a database interface with a 'Result Grid' tab. Below the tab, there is a single column header 'StudentName' in a table structure. The grid is currently empty, indicating no data was returned by the query.

Explanation

The query returned no records, indicating that all enrolled students have attendance records. No student was found without attending any session.

Query 9

Business Question

Compare course-wise student performance.

SQL Query:

SELECT c.CourseName, ROUND(AVG(ar.Score),2) AS AverageScore

FROM Course c

JOIN Batch b ON c.CourseID=b.CourseID

JOIN Enrollment e ON b.BatchID=e.BatchID

JOIN AssessmentResult ar ON e.EnrollmentID=ar.EnrollmentID

GROUP BY c.CourseName;

Output

Result Grid

Filter Rows:

Export

	CourseName	AverageScore
▶	Python Programming	83.75
	SQL & Database	82.33
	Data Science	83.00
	Web Development	83.50

Explanation

This query compares the average assessment scores across different courses to evaluate overall course performance.

Query 10

Business Question

Rank students according to assessment results.

SQL Query:

```
SELECT s.StudentName,
AVG(ar.Score) AS AverageScore,
RANK() OVER(ORDER BY AVG(ar.Score) DESC) AS StudentRank
FROM Student s
JOIN Enrollment e ON s.StudentID=e.StudentID
JOIN AssessmentResult ar ON e.EnrollmentID=ar.EnrollmentID
GROUP BY s.StudentName;
```

Output

Result Grid				Filter Rows:	Export:
	StudentName	AverageScore	StudentRank		
▶	Pooja Patil	95.000000	1		
	Sneha Kulkarni	91.000000	2		
	Manali Tamboli	88.666667	3		
	Amit Patil	88.000000	4		
	Priya Joshi	80.666667	5		
	Rohan Shah	75.000000	6		
	Rahul Jadhav	72.000000	7		
	Karan Deshmukh	69.000000	8		

Explanation

This query ranks students according to their average assessment scores using the RANK() window function.

Query 11

Business Question

Find batches with the highest and lowest average attendance.

SQL Query:

SELECT b.BatchName,

***ROUND(AVG(CASE WHEN a.Status='Present' THEN 100 ELSE 0
END),2) AS AverageAttendance***

FROM Batch b

JOIN Enrollment e ON b.BatchID=e.BatchID

JOIN Attendance a ON e.EnrollmentID=a.EnrollmentID

GROUP BY b.BatchName

ORDER BY AverageAttendance DESC;

Output

Result Grid			Filter Rows:	Export:
	BatchName	AverageAttendance		
▶	SQL Evening	100.00		
	Python Morning	75.00		
	Data Science Weekend	66.67		
	Web Development Morning	50.00		

Explanation

This query calculates the average attendance of each batch and helps identify batches with the highest and lowest attendance percentages.

Query 12

Business Question

Generate a student-wise training progress report.

SQL Query:

SELECT s.StudentName,

c.CourseName,

b.BatchName,

ROUND(AVG(ar.Score),2) AS AverageScore

FROM Student s

JOIN Enrollment e ON s.StudentID=e.StudentID

JOIN Batch b ON e.BatchID=b.BatchID

JOIN Course c ON b.CourseID=c.CourseID

JOIN AssessmentResult ar ON e.EnrollmentID=ar.EnrollmentID

GROUP BY s.StudentName,c.CourseName,b.BatchName;

Output

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
StudentName	CourseName	BatchName	AverageScore
Manali Tamboli	Python Programming	Python Morning	88.50
Priya Joshi	Python Programming	Python Morning	79.00
Amit Patil	SQL & Database	SQL Evening	88.00
Rohan Shah	SQL & Database	SQL Evening	75.00
Priya Joshi	SQL & Database	SQL Evening	84.00
Sneha Kulkarni	Data Science	Data Science Weekend	91.00
Karan Deshmukh	Data Science	Data Science Weekend	69.00
Manali Tamboli	Data Science	Data Science Weekend	89.00
Pooja Patil	Web Development	Web Development Morning	95.00
Rahul Jadhav	Web Development	Web Development Morning	72.00

Explanation

This query generates a complete progress report for each student by combining course, batch, attendance, and assessment information.

•

11. Findings

Update these based on your actual query results. For example:

- The database successfully stores and manages all training institute information.
- Course-wise enrollment can be analyzed easily.

- Attendance reports help identify students with low attendance.
- Assessment reports help identify high-performing students.
- The use of joins and aggregate functions makes report generation efficient.

12. Conclusion

The Training Institute Management System successfully demonstrates the use of MySQL for managing institutional data. The project applies relational database concepts such as primary keys, foreign keys, joins, aggregate functions, views, and window functions. It helps maintain organized records and generate meaningful reports for better decision-making. The system can be extended further by adding features such as fee management, certificates, or online attendance